

實習生月報

PDO 資料路徑重建

CoE / SDO 交易重建

PDO Mapping / Assignment 關聯整理

SyncManager / FMMU 資料路徑分析

報告期間

2026.08

MONTHLY REPORT

本月工作大綱



08/xx - 08/xx

理解封包與 EtherCAT 100%

- 使用 Wireshark 觀察從上電抓的原始 EtherCAT 封包
- 整理 Broadcast、AP、FP 等指令對 Slave 的操作方式
- 整理 Master 對 Slave ESC 暫存器的讀寫流程
- 理解 CoE、SDO、PDO 與 Object Dictionary 的關係
- 建立本月主線：CoE / SDO → PDO Mapping / Assignment → SyncManager → FMMU

08/xx - 08/xx

建立 Slave 拓樸關係 80%

- 理解 Master 如何透過 Auto Increment / Configured Address 指定 Slave
- 理解 Configured Station Address 與 Configured Station Alias 的差異
- 設計 AP Datagram 配對、Slave Discovery 模型與分析流程

設計 Analyzer 驗證 Agent 20%

- 建立 Stage 3 Result Check
- 由 Capture evidence 驗證 Slave Count、Topology、Address 與 Identity

08/xx - 08/xx

Mailbox / CoE / SDO 70%

- 在 Datagram Record 中加入 CoE / SDO decoded fields
- 重建 SDO Initiate Download Request / Response Transaction
- 依 SDO Index / SubIndex 辨識 PDO Mapping / Assignment 操作
- 重建 RxPDO / TxPDO Mapping Configuration
- 重建 0x1C12 / 0x1C13 PDO Assignment Configuration

完善 Agent 驗證流程 30%

- 整理 TShark EtherCAT / CoE / SDO 查詢與 JSON 結構規格
- 將常用 Capture 查詢封裝成 Python tools，提供 LLM 多種驗證方式

08/xx - 08/xx

SyncManager / FMMU 分析 100%

- 在 Datagram Record 中加入 SyncManager / FMMU decoded fields
- 重建 SyncManager Configuration
- 重建 FMMU Configuration
- 驗證 PDO Assignment 與 SM2 / SM3 的設定關係
- 驗證 FMMU Physical Address 與 SyncManager RAM 區段的對應

SECTION / 01

01 上電流程與 ESC Register 解析

追蹤啟動封包，建立 ESC 暫存器存取與解析脈絡

上電封包操作流程

01 Broadcast Initialization

BRD / BWR / BRW
All Slaves

02 Auto Increment Addressing

APRD / APWR / APRW
Topology Position

03 Configured Station Address

APWR / ADO 0x0010
Fixed Address

04 Fixed Position ESC Access

FPRD / FPWR / FPRW
Configured Address + ADO

Master

Slave 1

Outgoing ADP: 0x0000
Signed: 0
Topology Position: 1

Slave 2

Outgoing ADP: 0xFFFF
Signed: -1
Topology Position: 2

Slave 3

Outgoing ADP: 0xFFFE
Signed: -2
Topology Position: 3

Outgoing ADP

Signed Value

命中的拓樸位置

0x0000

0

Slave 1

0xFFFF

-1

Slave 2

0xFFFE

-2

Slave 3

Master 以 outgoing ADP = 0、-1、-2... 逐站指定 Slave。
ADO 是目標 ESC Register Offset，命中的 Slave 會執行該 Datagram。

SII EEPROM 讀取流程

01 檢查 EEPROM 狀態

- 讀取 ESC Register **0x0502~0x0503**
- 確認 Busy 清除且沒有 Error

02 指定 SII Word Address

- 寫入 ESC Register **0x0504~0x0507**
- **0x0008** = Vendor ID · **0x000A** = Product Code 起始 Word

03 送出 EEPROM Read Command

- 在 **0x0502~0x0503** 的 Control / Status 欄位送出 Read Command

04 讀取 EEPROM Data

- 以 APRD 讀取 **ADO = 0x0508**
- 依前面指定的 SII Word Address 解讀資料內容

Address Space	Address	Meaning
ESC Register	0x0502~0x0503	EEPROM Control / Status
ESC Register	0x0504~0x0507	EEPROM Address
ESC Register	0x0508~0x050F	EEPROM Data
SII EEPROM Word	0x0008~0x0009	Vendor ID
SII EEPROM Word	0x000A~0x000B	Product Code

實際 Capture

實際 Capture 中，Master 會以一個 **APWR** Datagram，從 **ADO = 0x0502** 開始一次寫入 **6 Bytes**。前 2 Bytes 對應 **0x0502~0x0503** 的 EEPROM Control / Status，後 4 Bytes 對應 **0x0504~0x0507** 的 EEPROM Address，因此一次 **APWR** 即可同時送出 Read Command 與欲讀取的 SII Word Address。

EEPROM Read 完成後，Master 再以 **APRD** 讀取 **ADO = 0x0508** 的 EEPROM Data。實際取得的資料內容，則由前面寫入的 SII Word Address 決定，例如 **0x0008** 對應 Vendor ID、**0x000A** 對應 Product Code。



Slave Discovery 重建成果

01 重點統整

AP Commands

02 AP-Analyzer 重建流程

Slave Discovery

03 程式碼實作

Class Relationship

04 實際成果

Stage 3 Result

Slave Discovery

| AP Datagram

| Topology Position

| Configured Address

| Slave Identity

CoE / SDO 知識整理

01 Mailbox / CoE / SDO
Protocol Layers

02 Object Dictionary
Index / SubIndex

03 SDO Download / Upload
Client / Server

04 SDO → PDO
Configuration
PDO Mapping

CoE / SDO 知識主線

EtherCAT Datagram

Mailbox

CoE

SDO

Object Dictionary

PDO Configuration

CoE / SDO 的作用是透過 EtherCAT Mailbox 存取 CANopen Object Dictionary，並可進一步用來設定 PDO Mapping / Assignment。

SDO Transaction / PDO Mapping 重建成果

EtherCATAnalyzer.TestConsole.exe analyze

Stage 4 / 105 Transactions

[Stage 4] SDO Transaction Analysis

Elapsed: 6.226 s

Transaction Count: 105

Configured Slave Address: 0x0001,	Object: 0x1C12:00,	Data: 0x00,	Request Frame: 41394,	Response Frame: 41403,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1C13:00,	Data: 0x00,	Request Frame: 41408,	Response Frame: 41417,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1A00:00,	Data: 0x00,	Request Frame: 41422,	Response Frame: 41455,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1A00:01,	Data: 0x60410010,	Request Frame: 41460,	Response Frame: 41614,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1A00:02,	Data: 0x60430010,	Request Frame: 41639,	Response Frame: 41786,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1A00:03,	Data: 0x30221110,	Request Frame: 41791,	Response Frame: 41958,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1A00:04,	Data: 0x30210c10,	Request Frame: 41963,	Response Frame: 42130,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1A00:05,	Data: 0x30210210,	Request Frame: 42135,	Response Frame: 42302,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1A00:00,	Data: 0x05,	Request Frame: 42307,	Response Frame: 42316,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1A01:00,	Data: 0x00,	Request Frame: 42321,	Response Frame: 42330,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1A02:00,	Data: 0x00,	Request Frame: 42335,	Response Frame: 42344,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1A03:00,	Data: 0x00,	Request Frame: 42349,	Response Frame: 42376,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1600:00,	Data: 0x00,	Request Frame: 42381,	Response Frame: 42496,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1600:01,	Data: 0x60420010,	Request Frame: 42501,	Response Frame: 42676,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1600:00,	Data: 0x01,	Request Frame: 42681,	Response Frame: 42690,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1601:00,	Data: 0x00,	Request Frame: 42695,	Response Frame: 42704,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1601:01,	Data: 0x60400010,	Request Frame: 42709,	Response Frame: 42842,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1601:02,	Data: 0x60600008,	Request Frame: 42847,	Response Frame: 43014,	Success: True,	Abort Code: N/A
Configured Slave Address: 0x0001,	Object: 0x1601:03,	Data: 0x60720010,	Request Frame: 43019,	Response Frame: 43193,	Success: True,	Abort Code: N/A



EtherCAT Spec 重點整理

01 KEY CONCEPT	Core concept or mechanism Summarize the core mechanism, state transition, or protocol rule that defines the behavior described in the specification. Note what the device or analyzer should observe when this rule is applied.
02 IMPORTANT FIELDS	Registers / objects / parameters Record register addresses, object indices, data types, access rules, and parameter constraints. Highlight which fields are read-only, writable, or dependent on operating state.
03 DATA FLOW	How data moves through the system Trace how the request, datagram, mapped memory, and application data move through the system. Mark the boundaries where timing, ownership, or synchronization affects the result.
04 ENGINEERING NOTE	Implementation or analyzer relevance Connect the specification detail to source code, packet analysis, validation steps, or observable system behavior. Record the implementation risk or evidence that should be checked.



EtherCAT 圖解分析

FIGURE / 04



[Diagram Canvas]

Class Diagram / Architecture Diagram / Data Path Figure

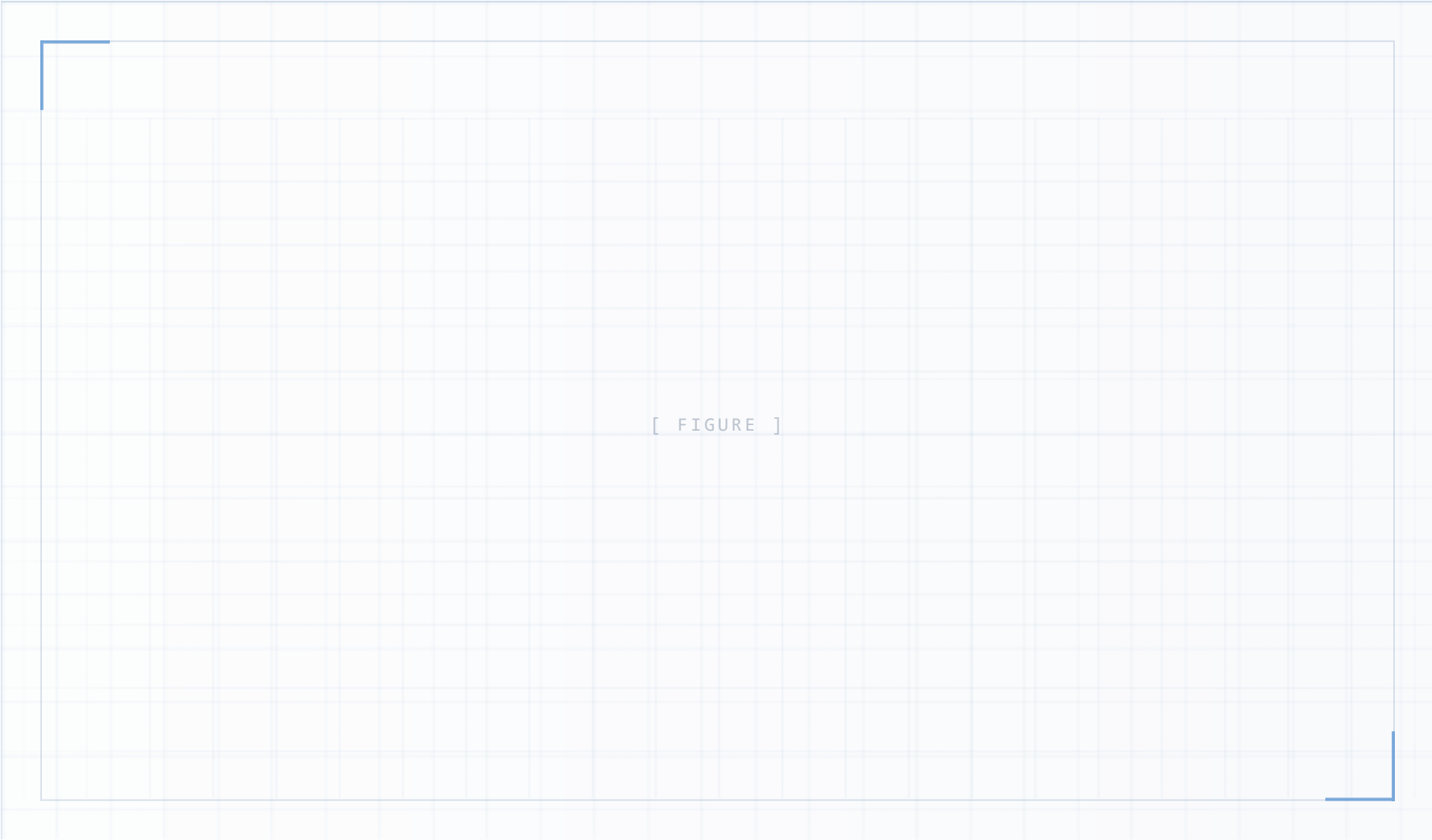
REPLACE WITH IMG / SVG / FIGURE / HTML DIAGRAM

FOCUS POINTS

- 01 Diagram Purpose**
Explain the main relationship between modules.
- 02 Key Components**
Highlight the major classes or architecture blocks.
- 03 Reading Order**
Describe how to read the diagram efficiently.
- 04 Implementation Relevance**
Connect the diagram to the current development work.



EtherCAT 資料路徑概覽



KNOWLEDGE POINTS

01

FMMU

Logical Address
→ ESC local

02

SyncManager

Consistent data
exchange control

03

Process RAM

Stores mapped
process data

04

DATA PATH

Logical Address → FMMU →
SM → RAM → Application

